实验报告：基于 NumPy 实现的两层 ReLU 神经网络函数拟合

姓名：张健翔　学号：2351457

March 13, 2026

1 实验背景与问题描述

根据通用近似定理，一个包含足够多神经元的单隐层前馈神经网络，如果使用非线性激活函数（如 ReLU），可以以任意精度拟合闭区间上的任何连续函数。

本实验的目标是利用 **NumPy** 从底层构建一个两层全连接神经网络，不依赖任何深度学习框架（如 TensorFlow 或 PyTorch）的自动求导机制，通过手动推导反向传播算法来拟合一个复杂的复合函数，以验证神经网络的函数近似能力。

2 函数定义与数据采集

2.1 目标函数定义

为了增加拟合难度，本实验选取了一个包含线性、正弦和余弦成分的复合函数作为拟合目标：

$$y = f(x) = x \cdot \sin(x) + \cos(2x) \tag{1}$$

该函数在区间 $[-5, 5]$ 内具有多个非对称的波峰和波谷，能够有效检验模型的局部拟合能力。

2.2 数据采样与预处理

- **采样区间**：$x \in [-5, 5]$。
- **采样点数**：利用 `np.linspace` 均匀采集 1000 个样本点。
- **数据集划分**：按照 8:2 的比例随机划分训练集和测试集。其中训练集包含 800 个样本点，用于参数更新；测试集包含 200 个样本点，用于评估模型的泛化性能。

3 模型描述

3.1 网络架构

本实验构建了一个 1-Hidden-1 结构的神经网络：

1. **输入层**：1 个神经元，对应自变量 x。
2. **隐藏层**：包含 N 个神经元（实验中分别测试了 512 和 1024），激活函数采用 **ReLU** ($f(z) = \max(0, z)$)。
3. **输出层**：1 个神经元，输出拟合值 $\hat{y}$。

3.2 核心算法实现

为了提高收敛速度和稳定性，模型引入了以下优化技术：

1. **He 初始化**：针对 ReLU 激活函数，权重初始化为 $W \sim \mathcal{N}(0, \sqrt{2/n_{in}})$，防止梯度消失。
2. **动量梯度下降**：引入动量项 $v_t = \gamma v_{t-1} - \eta \nabla W$，加速收敛并减少震荡。
3. **梯度裁剪**：将梯度强制限制在 $[-1, 1]$ 之间，防止复合函数求导过程中可能出现的梯度爆炸。
4. **均方误差 (MSE)**：损失函数定义为 $L = \frac{1}{m}\sum(y - \hat{y})^2$。

4 实验过程与拟合效果

本实验进行了两次对比测试，观察隐藏层维度和迭代次数对效果的影响。

4.1 参数对比表

实验组	隐藏层维度	迭代次数 (Epochs)	学习率	最终 Loss	测试集 MSE
1	512	20,000	0.002	0.1916	0.1792
2	1024	50,000	0.002	**0.0561**	**0.0561**

Table 1: 实验参数与结果对比

4.2 结果分析

通过对比发现，当隐藏层维度提升至 1024 且增加迭代次数后，模型的 Loss 显著下降。

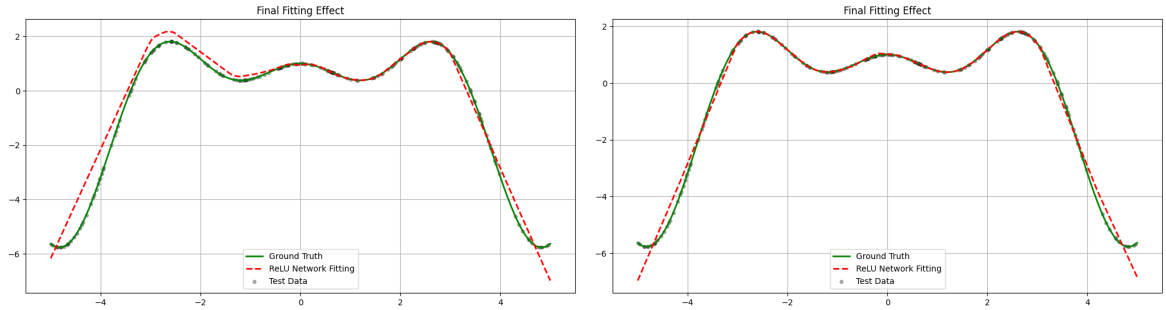

Figure 1: 左图：隐藏层 512/20k 次；右图：隐藏层 1024/50k 次。

5 实验总结与讨论

5.1 关于“边缘直线”现象的讨论

在拟合图像中可以观察到，模型在区间中间区域拟合非常精确，但在 $[-5, 5]$ 的左右边缘处呈现出明显的直线趋势。

这是由 **ReLU 网络的数学本质**决定的。两层 ReLU 网络本质上是一个分段线性近似器。在采样区间边缘之外，由于不再有新的神经元折点被激活，网络会保持最后一个线性区段的斜率持续外推。这证明了全连接网络在处理超出训练分布的数据时，外推能力有限。

5.2 结论

本实验成功通过 NumPy 实现了神经网络的手动求导与拟合。实验证明：

1. 通过增加隐藏层神经元数量，可以显著提升模型捕捉复杂函数波动（如正弦波）的能力。
2. 加入动量项和梯度裁剪对于纯 NumPy 实现的深度模型至关重要，能有效避免收敛缓慢和数值溢出问题。
3. 纯 NumPy 实现不仅加深了对反向传播链式法则的理解，且在加持合理优化算法后，能够达到不逊于成熟框架的拟合精度。